Your First ZipperGen Workflow

Draft a reply, ask a person, then send

The ZipperGen Authors

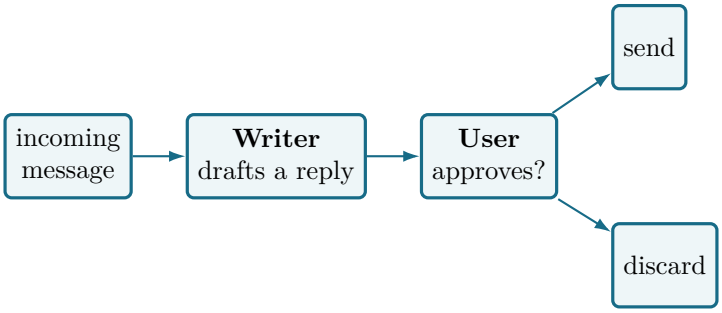
Tutorial edition: 8 August 2026

Abstract

You will build a small application that reads a message, asks a language model to draft a reply, and asks *you* to approve it before anything is sent. It is short enough to read in one sitting, and it already contains everything that makes ZipperGen worth using: two participants, a model call, an explicit human decision, and a branch that only the decider takes.

There is no ZipperGen application to learn. You work in an ordinary directory with an ordinary command line, and — if you like — a coding agent such as Claude Code or Codex. This tutorial shows both, and is honest about which parts need a person.

1 What you are building



Concretely, you want this to happen:

Incoming message

Could we move our meeting to Thursday afternoon?

Proposed reply

Thursday afternoon works for me. How about 3pm?

`[ Send ] [ Reject ]`

Two participants exchange messages, one of them is a person, and what happens next depends on what that person says. That is a coordination problem, and it is the kind of thing ZipperGen exists to make readable.

What you need

- Python 3.11 or newer and ZipperGen installed.
- Nothing else for the first two thirds. The model is mocked and the approval happens in your terminal.
- For the last section, a Telegram bot token, if you want the approval to arrive on your phone.

2 Create the project

```
mkdir email-approval && cd email-approval
zippergen init
```

```
ZipperGen project: email-approval
  zippergen.toml     created
  specification.md   created
  AGENTS.md          created
```

That is the whole of project creation. It writes three files and stops: it asks you nothing, and it configures nothing.

- `zippergen.toml` — the project's visible configuration. It is committed, and it never contains a credential.
- `specification.md` — what the workflow is meant to do, in prose.
- `AGENTS.md` — tells a coding agent to run `zippergen skill` before touching workflow code.

3 Describe what you want

You can write the workflow yourself; the next section shows the code. But the intended path is to say what you want in ordinary language. Open a coding agent in this directory:

```
claude
```

and ask for it:

```
> Build a ZipperGen workflow that takes an email message, asks an LLM to
  draft a short reply, and asks me to approve it before it is sent.
```

The agent reads `AGENTS.md`, which points it at `zippergen skill` — ZipperGen's own instructions for how to write a workflow, what to validate, and what never to touch. It then writes `specification.md` and `workflow.py`, and checks its work.

Where the skill comes from

`zippergen skill` prints instructions that ship inside the package, so they match the version you have installed. They are ordinary prose in the repository, not generated text — you can read them, and you can disagree with them.

4 What comes out

Visible, readable Python. This is the whole coordination:

```
@workflow
def email_approval(message: str @ User) -> str:
    User(message) >> Writer(message)
    Writer: draft = draft_reply(message)
    Writer(draft) >> User(draft)
    User: approved = approve_reply(draft)
    if approved @ User:
        User: result = sent(draft)
    else:
        User: result = discarded()
    return result @ User
```

Read it top to bottom and you have the entire message order. Four things are worth noticing.

`User(message) >> Writer(message)` is a message. The left side sends, the right side receives, and the names in brackets say which variables carry the value.

`Writer: draft = draft_reply(message)` is an action performed by one participant. `draft_reply` is an `@llm` action — a model call with a prompt and a declared output.

`User: approved = approve_reply(draft)` is a `@human` action. It stops and waits for a person.

`if approved @ User` is a decision, and the `@ User` says who makes it. That annotation is the important one, and the next section shows why.

5 Check it

```
zg validate
```

```
Workflow email_approval: valid
OK   lifelines: 2 unique lifeline(s): Writer, User
OK   projection Writer: SeqStmt
OK   projection User: SeqStmt
OK   deployment declaration: 0 field(s), 0 package(s), 0 setup step(s)
OK   canonical rendering: global and local code views rendered successfully
```

`validate` loads the module, projects the protocol for every participant, and renders it back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

6 The moment that matters

The workflow has one decision and the `User` makes it. Ask what each participant actually runs.

The `User` is the decider, so the branch is there:

```
zg show --agent User
```

```
@role('User')
def email_approval__User(message: str) -> str:
    send('Writer', message)
    draft = recv('Writer')
    approved = approve_reply(draft)
```

```
    if approved:
        result = sent(draft)
    else:
        result = discarded()
```

Now the `Writer`:

```
zg show --agent Writer
```

```
@role('Writer')
def email_approval__Writer() -> None:
    message = recv('User')
    draft = draft_reply(message)
    send('User', draft)
```

> **The Writer has no branch**
>
> Nobody wrote that. ZipperGen worked out that the `Writer` takes no part in the `User`'s decision, so the `Writer`'s local program does not mention it — and therefore cannot wait on it, disagree with it, or deadlock against it.
> In a hand-written multi-agent system this is where the bugs live: one agent waits for a message the other decided not to send. Here the question does not arise, because the projection removed it.

This is not a convenience. It is the construction the correctness proof is about: the projected local programs produce exactly the behaviours of the global protocol, and deadlock freedom follows for well-formed workflows.

7 Run it

No model key needed yet. `--llm mock` answers model calls with a placeholder:

```
zg run --llm mock
```

```
Message (str): Could we move our meeting to Thursday afternoon?

Proposed reply:

[draft_reply:draft]

Send this reply? [y/n]: y
{"result": "Sent: [draft_reply:draft]"}
```

The approval happened in your terminal, and answering `n` takes the other branch. The reply is a placeholder because `mock` does not call a model.

> **Testing a specific branch**
>
> `mock` answers every action the same way, so a workflow driven by it takes one path and no other. When you want to drive a particular branch — the rejection, or a loop running exactly three times — script the answers with `--llm scripted:replies.json`. The guide covers it; you do not need it yet.

8 Use a real model

Set the key in your environment and name a provider:

```
export OPENAI_API_KEY=sk-...
zg run --llm openai:gpt-4o-mini
```

There is no ZipperGen credential store to set up for development. The environment is where a key belongs, and ZipperGen reads it from there.

9 Approval on your phone

So far the approval appeared in your terminal. To send it to Telegram instead, the workflow declares that it needs a place to ask a human, and you say which chat that is.

```
zg connector configure telegram approvals --chat-id 12345678
```

```
Telegram bot token (input hidden):
Saved the Telegram bot token for this computer.
Saved connector configuration approvals (chat 12345678).
```

Two things went to two different places, on purpose:

chat id	written to `zippergen.toml`, committed, shared
bot token	typed, never an argument, stays on this computer

The token is read without echo, so it does not reach your shell history. A colleague who clones the project gets the chat id and supplies their own token.

> **This is the part that needs a person**
>
> A coding agent can write the workflow, validate it, and run it. It should not be given your credentials. Configuring a connector is deliberately something you do yourself, in your own terminal — which is the one moment in this tutorial where the two windows genuinely differ.

Coding agent

```
> Everything validates. To send
  approvals to Telegram, run this
  yourself:

    zippergen connector configure
      telegram approvals
      --chat-id <your chat id>

  I will not need the token.
```

Your terminal

```
$ zippergen connector configure
    telegram approvals
    --chat-id 12345678

Telegram bot token (hidden):
Saved the token for this computer.
Saved configuration approvals.
```

10 Runs that survive interruption

There is one verb for running a workflow. `--durable` records the run as it goes, so an interrupted run continues rather than starting over:

```
zg run --durable --llm mock
```

```
Workflow email_approval: valid
Run email_approval-20260808-132719-612865000
Proposed reply:
...
```

Press Ctrl-C while it waits for your approval, then:

```
zg run --resume
```

The model call that already happened is not repeated. The run resumes at the point it stopped, because every step was recorded before it was taken.

11 Deploy it

Preparing a deployment and starting one are separate steps, and the difference matters. Preparing writes files and reaches nothing:

```
zg deploy production --no-start
```

Starting is about to make real calls, so every model and connector is probed first, and a failure stops it:

```
zg start production
zg logs production
zg status production
```

A deployment you no longer want:

```
zg remove production
```

Its durable store is kept — that is the record of what actually ran, and it is the one thing no rebuild can recover. Everything else, the environment and service files and the bundled source, is written again by deploying again. `--purge` removes the store too, when you mean it.

12 What you have

`specification.md`	what it should do, in prose, maintained by you and your agent
`workflow.py`	the coordination, as visible Python
`zippergen.toml`	which model, which chat — committed, no secrets

Three files in a directory, plus whatever your machine supplies: a model key in the environment, a bot token typed once.

The thing worth remembering is the `Writer` with no branch. You wrote one protocol; ZipperGen derived what each participant runs, and derived it in a way that cannot deadlock. As the workflow grows — more participants, loops, parallel regions — that property is what stops the coordination becoming the hard part.

13 Where to go next

- `examples/diagnosis.py` — two reviewers argue until they agree. A loop, and a decision inside it.

- `examples/call_intake.py` — reads a real mailbox, records to a real spreadsheet, runs as a service.
- *Development and deployment guide* — the longer reference.
- `zippergen skill` — what your coding agent is following. Worth reading once yourself.